CSE513 Operating Systems

Assignment 1: CPU Scheduling Simulator

Website Address:	https://haruncakir.github.io/cpu_scheduler_sim/
Submission Date:	October 2026
Course:	CSE513 — Operating Systems
Institution:	Gebze Technical University

Live Website: The simulator is deployed and publicly hosted on GitHub Pages at https://haruncakir.github.io/cpu_scheduler_sim/. The project is built automatically using GitHub Actions upon pushing to the main branch.

Table of Contents

1. Introduction
2. Simulator Overview
3. Scheduling Algorithms
4. Required Workload Results
4.1 Workload A (Examples 1–4)
4.2 Workload B (Examples 5–8)
5. Additional Student-Designed Test Cases
6. Correctness Verification
7. Analysis (Questions 1–6)
8. Implementation Details
9. AI Assistance Statement
10. Conclusion
11. Submission Checklist

1. Introduction

This report documents the design, implementation, and verification of a browser-based CPU scheduling simulator produced for CSE513 Assignment 1. The simulator implements four classical CPU scheduling algorithms — First-Come, First-Served (FCFS), Shortest Job First (SJF), Shortest Remaining Time First (SRTF), and Round Robin (RR) — on a single-core CPU.

The application is built with TypeScript and Vite. The scheduling engine is a set of pure deterministic functions with zero DOM coupling. Simulation results are displayed through an interactive Gantt chart and a per-process metrics table.

This report presents independently hand-calculated scheduling results for both required workloads under all four algorithms, compares them with simulator output, and provides rigorous correctness verification. All simulator results are drawn from the actual running implementation and its 117 automated tests.

2. Simulator Overview

2.1 Architecture

The simulator is organised into two cleanly separated layers. The scheduling engine (src/scheduler/) contains pure functions that accept a process list and return a SimulationResult. The UI layer (src/main.ts, src/ui/) reads that result and renders DOM elements. No scheduling logic appears in the UI.

Layer	Modules	Responsibility
Scheduling Engine	types.ts, utils.ts, fcfs.ts, sjf.ts, srtf.ts, rr.ts, validate.ts	Pure functions. No DOM access.
User Interface	main.ts, ui/gantt.ts, ui/table.ts, index.html, style.css	Reads SimulationResult. Renders DOM. Zero scheduling logic.

2.2 Simulation Model

The simulator uses a **deterministic discrete-event model**. Time is measured in non-negative integers. The engine computes scheduling decisions analytically: for each selected process it finds the earliest event (arrival or quantum expiry) that could change the scheduling decision and jumps to that time. This avoids tick-by-tick loops and is correct for purely integer discrete time.

Each execution slice is stored as a half-open interval **[start, end)** where duration = end − start. A **null pid** denotes a CPU idle period. The timeline is always contiguous: interval[i].end = interval[i+1].start.

2.3 Metric Definitions

Metric	Formula	Meaning
Turnaround Time	completion − arrival	Total elapsed time from arrival to completion
Waiting Time	turnaround − burst	Time spent ready but not executing
Response Time	firstStart − arrival	Time from arrival to first CPU execution

3. Scheduling Algorithms

3.1 First-Come, First-Served (FCFS)

Non-preemptive. Processes are dispatched in order of arrival time. Ties resolved by position in the input list (inputIndex). CPU idle intervals are inserted when no process has arrived yet. **Implementation (fcfs.ts):** sorts once by (arrivalTime, inputIndex), then iterates linearly.

3.2 Shortest Job First (SJF) — Non-Preemptive

When the CPU becomes free, the ready process with the shortest CPU burst is selected. Once a process begins it runs to completion — arrival of shorter jobs does not interrupt it. Ties among ready processes: (burstTime, arrivalTime, inputIndex). **Implementation (sjf.ts):** at each dispatch point, filters arrived processes, sorts by (burst, arrival, index), picks the first.

3.3 Shortest Remaining Time First (SRTF) — Preemptive

The running process is preempted if and only if a newly arrived process has a **strictly shorter** remaining CPU time. Equal remaining times do **not** cause preemption. **Implementation (srtf.ts):** event-driven. For each selected process, computes its uninterrupted finish time, scans future arrival events within that window, and checks the strict condition *minArrivingBurst* < *remAtArr*.

3.4 Round Robin (RR) — Preemptive

Processes share the CPU in a FIFO ready queue with a configurable time quantum q. At a quantum boundary (time t = sliceEnd):

Step	Action
1	Enqueue all processes with arrivalTime ≤ sliceEnd not yet enqueued (sorted by arrivalTime, inputIndex).
2	If the running process has remaining work (rem > 0), re-enqueue it at the back.
3	Completed processes (rem = 0) are never re-enqueued.

Consecutive quantum slices of the same process are stored as distinct Interval objects so quantum boundaries remain visible in the Gantt chart. **Implementation (rr.ts):** enqueueArrivalsUpTo(sliceEnd) is called before queue.push(current).

3.5 Tie-Breaking Rule

All algorithms: when two processes are equally eligible, prefer the one with the **earlier arrival time**; if arrival times are equal, prefer the one with the **lower inputIndex** (earlier position in the input list). Implemented in the shared *tieBreak()* comparator in utils.ts.

4. Required Workload Results

This section contains all eight required worked examples. For each example, the hand-calculated result is shown first (derived independently of the simulator), followed by the simulator output and an explicit comparison. Notation: **P[s,e)** = process P runs from time s to e. **IDLE[s,e)** = CPU idle.

4.1 Workload A

Process	Arrival	CPU Burst
P1	0	8
P2	1	4
P3	2	2
P4	4	1

Example 1: FCFS — Workload A

Hand Calculation

FCFS serves in arrival order. P1 starts at t=0 and runs 8 units to t=8 (non-preemptive; P2, P3, P4 arrive during P1 but must wait). At t=8: all three others have arrived; P2 (arrived first, t=1) goes next. P2[8,12), P3[12,14), P4[14,15). No idle intervals.

```
P1[0,8) → P2[8,12) → P3[12,14) → P4[14,15)
```

Process	Arrival	Burst	1st Start	Completion	Turnaround	Waiting	Response
P1	0	8	0	8	8	0	0
P2	1	4	8	12	11	7	7
P3	2	2	12	14	12	10	10
P4	4	1	14	15	11	10	10
Average					10.50	6.75	6.75

Simulator source: fcfs.test.ts 'Workload A' (PASSING).

✓ Hand calculation matches simulator output exactly.

Example 2: SJF (Non-Preemptive) — Workload A

Hand Calculation

At t=0 only P1 is ready → P1 runs [0,8). **SJF is non-preemptive**: P2(t=1), P3(t=2), P4(t=4) all arrive during P1 but cannot interrupt it. At t=8, ready set: P2(burst=4), P3(burst=2), P4(burst=1). Shortest burst → P4 [8,9). Next: P3(burst=2) [9,11). Finally P2 [11,15).

```
P1[0,8) → P4[8,9) → P3[9,11) → P2[11,15)
```

Process	Arrival	Burst	1st Start	Completion	Turnaround	Waiting	Response
P1	0	8	0	8	8	0	0
P2	1	4	11	15	14	10	10
P3	2	2	9	11	9	7	7
P4	4	1	8	9	5	4	4
Average					9.00	5.25	5.25

Simulator source: sjf.test.ts 'Workload A' (PASSING).

✓ **Hand calculation matches simulator output exactly.**

Example 3: SRTF (Preemptive) — Workload A

Hand Calculation

Preemption fires only when a new arrival has **strictly** shorter remaining time.

Time	Event	Scheduling decision
t=0	P1 arrives	Only P1 ready → P1 starts. rem[P1]=8.
t=1	P2 arrives (burst=4)	4 < rem[P1]=7? YES → preempt P1 (rem[P1]=7). Select: P2(4)<P1(7) → P2 starts.
t=2	P3 arrives (burst=2)	2 < rem[P2]=3? YES → preempt P2 (rem[P2]=3). Select: P3(2)<P2(3)<P1(7) → P3 starts.
t=4	P3 done; P4 arrives (burst=1)	rem[P3]=0. Ready: P4(1),P2(3),P1(7). P4(1) shortest → P4[4,5).
t=5	P4 done	Ready: P2(rem=3),P1(rem=7). P2 resumes [5,8).
t=8	P2 done	Only P1 left (rem=7). P1[8,15).
t=15	P1 done	All complete. P1 total: [0,1)+[8,15)=1+7=8 units ✓

`P1[0,1) → P2[1,2) → P3[2,4) → P4[4,5) → P2[5,8) → P1[8,15)`

Process	Arrival	Burst	1st Start	Completion	Turnaround	Waiting	Response
P1	0	8	0	15	15	7	0
P2	1	4	1	8	7	3	0
P3	2	2	2	4	2	0	0
P4	4	1	4	5	1	0	0
Average					6.25	2.50	0.00

Simulator source: srtf.test.ts 'Workload A' (PASSING).

✓ **Hand calculation matches simulator output exactly.**

Example 4: Round Robin (q=2) — Workload A

Hand Calculation

Key rule at each quantum boundary: enqueue new arrivals BEFORE re-enqueuing the running process.

Step	Time Slice	rem after	Queue (front→back) after
1	P1[0,2) | arrivals≤2: P2(t=1). Re-enqueue P1.	P1=6	[P2, P1]
2	P2[2,4) | arrivals≤4: P3(t=2),P4(t=4). Re-enqueue P2.	P2=2	[P3, P1, P4, P2]
3	P3[4,6) | P3 done (burst=2, exactly one quantum).	P3=0	[P1, P4, P2]
4	P1[6,8) | Re-enqueue P1.	P1=4	[P4, P2, P1]
5	P4[8,9) | P4 done (burst=1, less than quantum).	P4=0	[P2, P1]
6	P2[9,11) | P2 done (rem=2, exactly one quantum).	P2=0	[P1]
7	P1[11,13) | Re-enqueue P1 (rem=2).	P1=2	[P1]
8	P1[13,15) | P1 done.	P1=0	[]

Note steps 7–8: P1 is the only remaining process and receives two consecutive slices [11,13) and [13,15). Each is stored as a *separate* Interval so quantum boundaries remain visible in the Gantt chart.

`P1[0,2)` → `P2[2,4)` → `P3[4,6)` → `P1[6,8)` → `P4[8,9)` → `P2[9,11)` → `P1[11,13)` → `P1[13,15)`

Process	Arrival	Burst	1st Start	Completion	Turnaround	Waiting	Response
P1	0	8	0	15	15	7	0
P2	1	4	2	11	10	6	1
P3	2	2	4	6	4	2	2
P4	4	1	8	9	5	4	4
Average					8.50	4.75	1.75

Simulator source: rr.test.ts 'Workload A q=2' (PASSING).

✓ Hand calculation matches simulator output exactly.

4.2 Workload B

Process	Arrival	CPU Burst
P1	2	3
P2	2	1
P3	8	2
P4	9	1

Example 5: FCFS — Workload B

Hand Calculation

No process arrives before t=2 → IDLE[0,2). At t=2, P1 and P2 both arrive. FCFS tie-break by inputIndex: P1(idx=0) first. P1[2,5). P2[5,6). Nothing ready in [6,8) → IDLE[6,8). P3[8,10). P4 arrives at t=9 while P3 runs (non-preemptive) → P4[10,11).

```
IDLE[0,2) → P1[2,5) → P2[5,6) → IDLE[6,8) → P3[8,10) → P4[10,11)
```

Process	Arrival	Burst	1st Start	Completion	Turnaround	Waiting	Response
P1	2	3	2	5	3	0	0
P2	2	1	5	6	4	3	3
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1
Average					2.75	1.00	1.00

Simulator source: fcfs.test.ts 'Workload B' (PASSING).

✓ Hand calculation matches simulator output exactly.

Example 6: SJF (Non-Preemptive) — Workload B

Hand Calculation

IDLE[0,2). At t=2: P1(burst=3) and P2(burst=1) ready. SJF picks shortest → P2[2,3). At t=3: only P1 ready → P1[3,6). IDLE[6,8). P3[8,10). P4 arrives t=9, P3 running non-preemptively → P4[10,11).

```
IDLE[0,2) → P2[2,3) → P1[3,6) → IDLE[6,8) → P3[8,10) → P4[10,11)
```

Process	Arrival	Burst	1st Start	Completion	Turnaround	Waiting	Response
P1	2	3	3	6	4	1	1
P2	2	1	2	3	1	0	0
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1
Average					2.25	0.50	0.50

Simulator source: sjf.test.ts 'Workload B' (PASSING).

✓ Hand calculation matches simulator output exactly.

Example 7: SRTF (Preemptive) — Workload B

Hand Calculation

IDLE[0,2). At t=2, CPU was idle — SRTF dispatches: P2(burst=1) < P1(burst=3) → P2 first. P2[2,3). At t=3, only P1 ready (rem=3). No shorter arrival before t=6 → no preemption. P1[3,6). IDLE[6,8). P3[8,10). At t=9: P4 arrives with burst=1, P3 rem=1. **1 is NOT strictly less than 1** → no preemption. P3 completes [8,10). P4[10,11).

Key observation: SRTF and SJF produce the identical schedule for Workload B. The only potential preemption (P4 at t=9 vs P3 rem=1) is blocked by the equal-remaining rule. This is verified by test: srtf.test.ts 'Workload B (same as SJF)'.

`IDLE[0,2) → P2[2,3) → P1[3,6) → IDLE[6,8) → P3[8,10) → P4[10,11)`

Process	Arrival	Burst	1st Start	Completion	Turnaround	Waiting	Response
P1	2	3	3	6	4	1	1
P2	2	1	2	3	1	0	0
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1
Average					2.25	0.50	0.50

Simulator source: srtf.test.ts 'Workload B' (PASSING).

✓ Hand calculation matches simulator output exactly.

Example 8: Round Robin (q=2) — Workload B

Hand Calculation

IDLE[0,2). At t=2: enqueue P1(idx=0) then P2(idx=1) → queue=[P1,P2]. P1[2,4) rem=1. Arrivals≤4: none. Re-enqueue P1. queue=[P2,P1]. P2[4,5): slice=min(2,1)=1. P2 done. queue=[P1]. P1[5,6): rem=1, done. queue empty. IDLE[6,8). Enqueue P3(t=8). P3[8,10) done. Enqueue P4(t=9). P4[10,11) done.

`IDLE[0,2) → P1[2,4) → P2[4,5) → P1[5,6) → IDLE[6,8) → P3[8,10) → P4[10,11)`

Process	Arrival	Burst	1st Start	Completion	Turnaround	Waiting	Response
P1	2	3	2	6	4	1	0
P2	2	1	4	5	3	2	2
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1
Average					2.75	1.00	0.75

Simulator source: rr.test.ts 'Workload B q=2' (PASSING).

✓ Hand calculation matches simulator output exactly.

5. Additional Student-Designed Test Cases

Three workloads were designed to exercise specific scheduling rules. They are defined in integration.test.ts and tested against all algorithm variants.

5.1 Custom Workload 1 — CPU Idle Time

Process	Arrival	CPU Burst
P1	3	2
P2	8	4
P3	16	1

Purpose: Three processes with large inter-arrival gaps ensure multiple idle intervals. This tests that the scheduler correctly inserts null-pid intervals when no process is ready.

Rule under test: CPU must remain idle until the next arrival. No process may execute before its arrivalTime.

Expected FCFS result:

```
IDLE[0,3) → P1[3,5) → IDLE[5,8) → P2[8,12) → IDLE[12,16) → P3[16,17)
```

Actual simulator result: Matches expected for all algorithm variants.

Invariant verified: No idle interval may contain a time t at which an arrived, incomplete process exists. Checked programmatically in integration.test.ts (invariant 5) for FCFS, SJF, SRTF, RR(q=1), RR(q=2), RR(q=4) — all 6 pass.

5.2 Custom Workload 2 — Simultaneous Arrivals and Scheduling Tie

Process	Arrival	CPU Burst
P1	0	3
P2	0	3
P3	0	2
P4	0	2

Purpose: All four processes arrive at t=0 with two identical burst pairs. Forces tie-breaking by inputIndex when burst lengths (for SJF/SRTF) or arrival times (for FCFS) are equal.

Rule under test: Tie-breaking: earlier arrivalTime first, then lower inputIndex.

Expected results:

```
[FCFS]: P1[0,3) → P2[3,6) → P3[6,8) → P4[8,10) (inputIndex order) [SJF/SRTF]: P3[0,2) → P4[2,4) → P1[4,7) → P2[7,10) (burst first, then inputIndex)
```

Actual simulator result: Matches expected for all variants. Verified by integration.test.ts invariant tests and adversarial.test.ts TC-B.

5.3 Custom Workload 3 — SRTF Preemption Chain

Process	Arrival	CPU Burst
P1	0	10
P2	2	5
P3	4	1

Purpose: P1 has a long burst. P2 arrives and triggers preemption (5 < rem[P1]=8). P3 arrives during P2's run and triggers a second preemption (1 < rem[P2]=3). Tests correct state accounting across a chain of two preemption events.

Rule under test: Preemption fires only when strictly shorter. Remaining times must be tracked accurately across multiple preemptions.

Time	Event	Action
t=0	P1 arrives	P1 starts. rem[P1]=10.
t=2	P2 arrives (burst=5)	5 < rem[P1]=8 → preempt. rem[P1]=8. Select P2(5<8). P2[2,4).
t=4	P3 arrives (burst=1)	1 < rem[P2]=3 → preempt. rem[P2]=3. Select P3(1<3<8). P3[4,5).
t=5	P3 done	Select: P2(rem=3) < P1(rem=8). P2[5,8).
t=8	P2 done	Only P1 (rem=8). P1[8,16). Total P1: 2+8=10 units ✓

`P1[0,2)` → `P2[2,4)` → `P3[4,5)` → `P2[5,8)` → `P1[8,16)`

Actual simulator result: integration.test.ts workloadPreempt and adversarial.test.ts 'SRTF three-way preemption chain' both confirm this timeline (PASSING).

6. Correctness Verification

6.1 Test Suite

Test File	Tests	Scope
fcfs.test.ts	6	FCFS: Workloads A&B, idle gap, simultaneous arrivals, single process
sjf.test.ts	4	SJF: Workloads A&B, tie-breaking, single process
srtf.test.ts	6	SRTF: Workloads A&B, TC-3 preemption, TC-4 equal-remaining rule, idle
rr.test.ts	7	RR: Workloads A&B (q=2), boundary ordering, completion-at-boundary, consecutive slices, q=1
validate.test.ts	9	All input validation rules
utils.test.ts	6	tieBreak() and computeMetrics() unit tests
ui.test.ts	2	UI helper functions
integration.test.ts	30	5 invariants × 5 workloads × 6 algorithm variants
adversarial.test.ts	47	47 adversarial edge-case tests (post-implementation review)
TOTAL	117	All 117 tests pass

6.2 Scheduling Invariants

Invariant	How Verified	Result
Every process receives exactly its CPU burst	Sum of intervals = burstTime	✓ Test
No process runs before its arrival	interval.start ≥ proc.arrivalTime	✓ Test
Processes do not overlap on single CPU	Timeline contiguous; no gaps/overlaps	✓ Test
CPU never idle while a process is ready	Idle interval checked against all arrived incomplete procs	✓ Test
Waiting ≥ 0 and Response ≥ 0	Checked per metric	✓ Test
Turnaround = completion – arrival	Checked per metric	✓ Test
Waiting = turnaround – burst	Checked per metric	✓ Test
Response = firstStart – arrival	Checked per metric	✓ Test
SRTF preempts only when strictly shorter	Source: minArrivingBurst < remAtArr; TC-4 test	✓ Test + Inspect
RR: arrivals enqueued before running process re-enqueued	Source: enqueueArrivalsUpTo() before queue.push()	✓ Test + Inspect
RR: completed processes not re-enqueued	Source: if (newRem > 0) queue.push(current)	✓ Test + Inspect
Gantt chart = simulation timeline	UI calls renderGanttChart(result.timeline) directly	✓ Inspect

6.3 Verification Categories

Category	Evidence
Verified by automated test	117/117 tests pass. Algorithm outputs, metric formulas, and all invariants are checked.
Verified by manual calculation	All 8 required examples hand-calculated before consulting the simulator. Results match in all cases.
Verified by source inspection	SRTF preemption condition (< not ≤), RR queue ordering, metric formula wiring in computeMetrics(), absence of scheduling logic i
Verified by browser, local production preview, and live GitHub Pages deployment (https://haruncakir.github.io/cpu_scheduler_sim/). All UI features confi	Tested by developer, executed

7. Analysis

Question 1: Best Average Waiting and Response Times for Workload A

Algorithm	Avg Turnaround	Avg Waiting	Avg Response
FCFS	10.50	6.75	6.75
SJF (non-preemptive)	9.00	5.25	5.25
SRTF	6.25	2.50	0.00
Round Robin (q=2)	8.50	4.75	1.75

Lowest average waiting time: SRTF (2.50). SRTF continuously selects the process with the least remaining work, preventing short jobs from waiting behind long ones. P3(burst=2) and P4(burst=1) are served almost immediately after arrival. In FCFS, P2, P3, P4 must wait behind P1's entire 8-unit burst before any of them start, leading to large accumulated waits.

Lowest average response time: SRTF (0.00). Every process in Workload A starts exactly at its arrival time: P1 at t=0 (only ready process), P2 at t=1 (preempts P1 immediately since 4<7), P3 at t=2 (preempts P2 since 2<3), P4 at t=4 (P3 just completed; P4 dispatched immediately). Response = firstStart − arrival = 0 for all four. RR achieves a good response time (1.75) because time-sharing gives each process its first slice quickly, but P3 and P4 still wait one full quantum each.

Question 2: Effect of RR Quantum Size on Workload A

Quantum	Avg Turnaround	Avg Waiting	Avg Response	Observation
q=1	8.50	4.75	0.75	Finest interleaving. Best response. Same wait/TT as q=2 for this workload.
q=2	8.50	4.75	1.75	Good balance. Standard textbook example.
q=4	9.25	5.50	3.75	P2,P3,P4 wait a full quantum before 1st service → worst response.

q=1 gives the best average response time (0.75): processes interleave after every unit and receive their first slice sooner. However, a smaller quantum does not always improve every metric — for this workload, q=1 and q=2 produce the same average turnaround and waiting because the burst lengths happen to align well with both quanta.

q=4 degrades response time (3.75) and waiting time (5.50) because P2 must wait for P1's entire first quantum [0,4) before its first slice, and P3 and P4 wait even longer. In a real OS, very small quanta increase context-switch overhead; very large quanta approach FCFS behaviour. A quantum between average and maximum burst is usually a practical compromise.

Question 3: When Do SJF and SRTF Produce the Same Schedule?

SJF and SRTF produce identical schedules when **no process with a strictly shorter burst arrives while the CPU is occupied**. This happens when:

1. All shorter-burst processes arrive only while the CPU is idle (at dispatch points).

2. No arrival event during any execution window has burst strictly less than the running process's remaining time.

Concrete example — Workload B: P2(arrival=2, burst=1) and P1(arrival=2, burst=3) arrive simultaneously at t=2 when the CPU is idle. Both algorithms dispatch P2 first (shortest burst at the dispatch point). After P2 completes at t=3, P1 is the only ready process and runs uninterrupted. The next candidate preemption is P4(t=9, burst=1) vs P3(rem=1) — but 1 is not strictly less than 1, so SRTF does not preempt. Result: identical schedules.

Question 4: Why Must a Real OS Estimate CPU Bursts?

In this simulator, every process's CPU burst is given as input before simulation starts. The scheduler has perfect foreknowledge. This is feasible in a controlled simulation but impossible in a real operating system.

In a real system, a process's next CPU burst depends on runtime behaviour: which code path is taken, which system calls are made, how the user interacts with the application, how long blocking I/O takes. None of this is known before

the process actually executes. A real kernel cannot query a running process for its future CPU usage.

Real SJF/SRTF implementations predict the next burst using historical data, typically via **exponential averaging**: next_estimate = $\alpha \times$ last_burst + $(1-\alpha) \times$ prior_estimate. This approximates well for processes with stable behaviour but introduces error for irregular workloads. The simulator bypasses this complexity using exact values, making it useful for comparing ideal algorithm behaviour without estimation noise.

Question 5: Can Starvation Occur Under SJF or SRTF?

Yes, in an open system with continuous arrivals. If short-burst processes keep arriving, a long-burst process may never be selected by SJF (it never reaches the shortest position) or may be continuously preempted by SRTF, accumulating unbounded waiting time. This is a well-known weakness of shortest-job-based algorithms.

In this assignment's finite workloads, starvation cannot occur: the process set is fixed. Once all shorter processes complete, the longest process will inevitably be selected. The simulation always terminates when all processes complete.

The standard remedy in real systems is **ageing**: a waiting process's effective priority increases gradually over time, eventually making it competitive regardless of its burst length.

Question 6: Implementation Mistake During Development

Honest disclosure: No genuine implementation bug in the scheduling engine was recorded during development. The adversarial verification pass (117 tests, manual traces, source inspection) found all four algorithms to be correct. Two errors were found in the test file itself — not in the implementation:
1. A wrong arithmetic expected value in the SRTF three-way preemption chain test (expected P1 end=15, correct value is end=16; P1 executes [0,2)+[8,16)=2+8=10 units). The implementation produced the correct value; the test expectation was wrong.
2. Six CommonJS require() calls inside test bodies, incompatible with verbatimModuleSyntax; corrected by converting to a top-level ES import.

Student action required: If a genuine implementation mistake was observed during development that is not documented above, describe it here before submission.

8. Implementation Details

8.1 Technology Stack

Component	Technology	Version
Language	TypeScript (strict mode, verbatimModuleSyntax)	~5.8.3
Build Tool	Vite	^6.3.5
Test Framework	Vitest	^4.1.11
Coverage	@vitest/coverage-v8	^4.1.11
Runtime	Browser (no server-side code)	—

8.2 Key Commands

Install dependencies:

```
npm install
```

Development server:

```
npm run dev → http://localhost:5173
```

Run all tests (117):

```
npm test
```

Production build:

```
npm run build → dist/ (12 kB JS, 0 errors)
```

Type-check only:

```
npx tsc --noEmit
```

8.3 UI Features

Feature	Implementation
Add process	'+ Add Process' button appends an editable row
Edit process	All fields (ID, arrival, burst) are editable inline
Remove process	'Remove' button on each row deletes it
Load Workloads A & B	Dedicated buttons populate the table with the assignment workloads
Select algorithm	Dropdown: FCFS, SJF, SRTF, Round Robin
Configure RR quantum	Quantum field visible only when RR is selected
Run simulation	Validates input, calls engine, renders results
Gantt chart	Proportional flexbox; each Interval = one block; idle shown as grey 'Idle'
RR quantum boundaries	Each slice is a separate Interval → separate block in Gantt
Per-process metrics	Table: ID, Arrival, Burst, 1st Start, Completion, Turnaround, Waiting, Response
Average metrics	Summary footer row: avg Turnaround, Waiting, Response
Invalid input handling	Error box with bulleted list of specific error messages; results hidden

9. AI Assistance Statement

9.1 Tools and Purposes

Phase	Tool/Skill	Purpose
Requirements	spec-driven-development skill	Analyzed the assignment PDF; produced requirements spec and architecture doc
Planning	planning-and-task-breakdown skill	Broke implementation into small verifiable increments
Implementation	incremental-implementation skill	Each module implemented in order, tests before advancing
Testing	test-driven-development skill	Tests written first (RED → GREEN) for each algorithm
Verification	code-review + debugging skills	Post-implementation adversarial review; 47 edge-case tests added

9.2 How AI-Generated Output Was Verified

AI-generated code was not accepted without independent verification. The following checks were performed:

1. Automated tests: 117 tests covering unit behavior, cross-algorithm invariants, and adversarial edge cases. Every algorithm function tested with exact expected values for Workloads A and B from the assignment PDF.

2. Manual scheduling calculations: all 8 required algorithm × workload examples were calculated by hand before comparing with simulator output (as documented in Section 4).

3. Source code inspection: the SRTF preemption condition (< not ≤), RR queue ordering, and metric formula wiring were explicitly confirmed in the source.

4. Build verification: tsc + vite build reported 0 errors and 0 warnings.

9.3 A Suggestion That Was Corrected

During the adversarial review phase, an AI-generated test expectation was wrong: the SRTF three-way preemption chain test expected P1 to finish at t=15, but the correct value is t=16 (P1 executes [0,2) + [8,16) = 2+8 = 10 units = its burst). The implementation produced the correct end=16; only the test expected value was wrong. This was caught by examining the failing test output, performing the arithmetic by hand, and confirming against the invariant that every process must receive exactly its burst.

10. Conclusion

The CSE513 Assignment 1 CPU Scheduling Simulator correctly implements all four required scheduling algorithms (FCFS, SJF, SRTF, and Round Robin) as a browser-based TypeScript application. The scheduler uses a deterministic discrete-event simulation model: scheduling decisions are computed analytically rather than through real-time execution, which is appropriate for the assignment's integer discrete-time model.

All eight required algorithm × workload examples were verified by hand calculation and by running the simulator; results match in every case. 117 automated tests covering unit behaviour, cross-algorithm invariants, and adversarial edge cases all pass, and the production build is clean.

The analysis confirms that SRTF achieves the lowest average waiting time and the best average response time for Workload A, as it continuously keeps the shortest-remaining job on the CPU. Round Robin (q=2) provides a good compromise between response time and throughput. FCFS, while simple, produces the worst average waiting time because it cannot reorder processes by burst length.

11. Submission Checklist

Item	Status	Notes
PDF report	✓	This document: cse513_report.pdf
Working website address	✓	https://haruncakir.github.io/cpu_scheduler_sim/ (GitHub Pages)

Item	Status	Notes
Complete source code	✓	hw1/ directory. ZIP or repo link provided.
README	✓	README.md with live URL, architecture, install, run, test instructions
Locally runnable simulator	✓	npm install && npm run dev → http://localhost:5173
AI-use statement	✓	Section 9 of this report
8 worked examples	✓	Section 4: FCFS, SJF, SRTF, RR × Workloads A & B
3 additional test cases	✓	Section 5: CPU idle, simultaneous arrivals, SRTF preemption chain
6 analysis questions	✓	Section 7: Questions 1–6 answered
Hand calculations before simulator output	✓	All 8 examples show hand calc first, then simulator output, then comparison
117 automated tests passing	✓	npm test: 117 tests, 9 files, all passed
Clean build	✓	npm run build: 0 TypeScript errors, 0 warnings

Hosted Deployment: The simulator is publicly available at https://haruncakir.github.io/cpu_scheduler_sim/. All submission checklist items are verified and complete.